

Chapter 8

Github Repository Link - <https://github.com/reim7/CS4080-HW4>

1. The REPL no longer supports entering a single expression and automatically printing its result value. That's a drag. Add support to the REPL to let users type in both statements and expressions. If they enter a statement, execute it. If they enter an expression, evaluate it and display the result value.

The REPL can be improved so that it supports both statements and expressions. When the user enters a statement, the interpreter should execute it normally. However, if the user enters an expression by itself, the interpreter should evaluate the expression and display its result without requiring a print statement.

To implement this, I modified the parser so that it can recognize when the final input is an expression without a semicolon. I also added an interpreter method that evaluates an expression and returns its value. The REPL then checks whether the parsed input is a statement or an expression. Statements are executed normally, while expressions are evaluated and their results are displayed.

This makes the REPL more convenient for quickly testing expressions.

2. Maybe you want Lox to be a little more explicit about variable initialization. Instead of implicitly initializing variables to `nil`, make it a runtime error to access a variable that has not been initialized or assigned to, as in:

```
// No initializers.

var a;

var b;

a = "assigned";

print a; // OK, was assigned first.

print b; // Error!
```

Normally, Lox initializes a variable to nil when no initializer is provided. I changed this behavior so that a variable can be declared without immediately receiving a value, but attempting to access that variable before assigning it a value causes a runtime error.

I implemented this using a special internal uninitialized value. When a variable is declared without an initializer, the interpreter stores this value instead of nil. Whenever a variable is accessed, the interpreter checks for the special value. If it is still uninitialized, a runtime error is produced.

One disadvantage is that every variable access now requires an additional check to determine whether the variable has been initialized.

3. What does the following program do?

```
var a = 1;

{

  var a = a + 2;

  print a;

}
```

What did you *expect* it to do? Is it what you think it should do? What does analogous code in other languages you are familiar with do? What do you think users will expect this to do?

When the initializer `a + 2` is evaluated, the new inner `a` has not been defined yet. Therefore, the lookup continues into the enclosing scope and finds the outer `a`, whose value is 1. The expression becomes `1 + 2`, so the new inner `a` receives the value 3. Lox's environments search enclosing scopes when a name is not found in the current environment.

In other languages, this situation may be handled differently. For example, Java does not allow a local variable to redeclare another local variable from an enclosing local scope in this way, so analogous code would result in a compile-time error. Lox is different because it allows a variable in an inner scope to shadow a variable with the same name in an outer scope.

At first, I would not necessarily expect this behavior because the same variable name appears on both sides of the declaration. It can look like `a` is referring to itself. However, after considering how Lox handles scope, the result makes sense because the inner `a` does not exist until after its initializer has been evaluated.

I think treating this situation as an error could be a clearer design. Referencing a variable with the same name while that variable is being initialized can be confusing. Preventing it would make it easier to catch cases where a programmer accidentally tries to use a variable before it has been fully declared.

In that design, the interpreter could mark the new local variable as declared but unavailable while its initializer is being evaluated. If the initializer attempts to access that same variable, Lox could report an error.

I think many users would find the code ambiguous at first. Some may expect `a + 2` to use the outer `a`, producing 3, while others may think it is incorrectly trying to access the new `a` before initialization.

Because both interpretations are reasonable at first glance, reporting an error may be less surprising and could encourage programmers to write clearer code. If the programmer wants to use the outer value, using a different variable name would make that intention obvious.

Chapter 9

1. **A few chapters from now, when Lox supports first-class functions and dynamic dispatch, we technically won't *need* branching statements built into the language. Show how conditional execution can be implemented in terms of those. Name a language that uses this technique for its control flow.**

Conditional execution can be implemented using first-class functions and dynamic dispatch instead of having the interpreter directly implement an if statement. The idea is to represent the two possible branches as functions and let Boolean objects decide which function should be executed.

For example, true and false could each provide an ifThenElse() method. The true object would execute the function representing the true branch, while the false object would execute the function representing the false branch. Since the branches are functions, the branch that is not selected does not need to execute.

Smalltalk is an example of a language that uses this general technique. Its Boolean objects determine which block of code to evaluate.

2. **Likewise, looping can be implemented using those same tools, provided our interpreter supports an important optimization. What is it, and why is it necessary? Name a language that uses this technique for iteration.**

Looping can be implemented using first-class functions and recursion. Instead of having a built-in loop repeatedly execute a block of code, a function can perform one iteration and then call itself to perform the next iteration.

The important optimization needed for this technique is tail-call optimization, also known as proper tail calls.

It is necessary because ordinary recursive calls normally create a new stack frame every time the function calls itself. A loop might execute thousands or even millions of times,

so without tail-call optimization, the program would eventually run out of stack space and cause a stack overflow.

With proper tail calls, when the recursive call is the final operation performed by the function, the interpreter can reuse the existing stack frame instead of creating a new one. This allows recursion to perform repeated execution without continually increasing the call stack.

Scheme is an example of a language that guarantees proper tail calls and can therefore use recursion effectively for looping.

3. Unlike Lox, most other C-style languages also support `break` and `continue` statements inside loops. Add support for `break` statements.

The syntax is a `break` keyword followed by a semicolon. It should be a syntax error to have a `break` statement appear outside of any enclosing loop. At runtime, a `break` statement causes execution to jump to the end of the nearest enclosing loop and proceeds from there. Note that the `break` may be nested inside other blocks and `if` statements that also need to be exited.

I added a break statement to Lox using the syntax. First, I added BREAK as a new token type and added "break" to the scanner's keyword list. I then added a Break statement to the AST and modified the parser so that it recognizes break;.

To make sure break is only used inside a loop, the parser keeps track of the current loop depth. When it begins parsing a while or for loop body, the loop depth increases, and when it finishes, the loop depth decreases. If the parser encounters a break when the loop depth is zero, it reports a syntax error.

At runtime, executing a break throws a special internal exception. The nearest enclosing loop catches that exception and stops executing. Using an exception allows the break to move through other nested statements and blocks until it reaches the closest loop.

